
Who Wrote This Paper? Fully Automated 3DGS Research via Multi-Agent Orchestration

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Autonomous scientific discovery promises to shorten the cycle from idea to pub-
2 lication, but existing pipelines such as AI Scientist (??), Agent Laboratory (?),
3 MLR-Copilot (?), CycleResearcher (?), AgentRxiv (?), and AI-Researcher (?)
4 stop short of publication-ready output: they evaluate on toy tasks, write papers
5 whose numbers are not mechanically traceable to experiments, and offer no way to
6 falsify a generated method inside the optimisation loop. We present IDEA2PAPER,
7 a fully automated pipeline for 3D Gaussian Splatting (3DGS) research that turns
8 a one-line prompt or a target-paper URL into a publication-ready NeurIPS-style
9 paper with real ns-train-measured PSNR, SSIM, and LPIPS on Mip-NeRF 360
10 and LLFF. The pipeline orchestrates 20+ specialised agents across seven stages
11 under a three-level academic hierarchy (a master professor agent, PhD-student
12 phase coordinators, and undergraduate leaf agents) and contributes five novelties
13 relative to prior autonomous-research and program-search systems (?????): (i) a
14 *taxonomy of seven 3DGS novelty categories* (loss and regularisation, densification
15 and pruning, rendering and rasterisation, parameter representation, optimisation
16 and scheduling, scaffold and structural priors, and geometric and frequency priors),
17 used as a structured palette that the ideator draws from either to improve an existing
18 method or to propose a new one, paired with a category-aligned EVOLVE-BLOCK
19 code scaffold and a cross-codebase auto-mapper that transfers the scaffold to any
20 SOTA 3DGS repository without per-repo annotation; (ii) a *hybrid evolutionary*
21 *search* that unifies EoH rationale-implementation pairs with an operator-level
22 Elo bandit, a grounded Elo ladder driven by real GPU PSNR with margin-aware
23 K scaling, and ReEvo-style verbal gradients restricted to surprising pairs and
24 partitioned by category; (iii) an *adversarial falsification* stage with three attacks
25 (edge scene, sparse view, hyper-parameter ablation with load-bearing crash-as-pass
26 semantics) whose outcomes subtract Elo, making robustness a first-class fitness
27 signal; (iv) a *hierarchical master orchestrator* with two-tier context compaction (a
28 pinned summary of the run’s early history that is never rewritten, plus incremental
29 delta summaries) that keeps the system on task across multi-day runs, backed by
30 a shared cross-run literature library that converts repeated crawls into a bounded
31 top-up, with an optional lab-configurable budget enforced proportionally across
32 phases; and (v) *claim-backed verification* as a stop-condition gate, so the system
33 cannot ship a paper whose numerical claims were not actually measured. We show
34 that IDEA2PAPER produces experimentally grounded papers whose methods match
35 or exceed strong 3DGS baselines while keeping every reported number traceable
36 to a real measurement.

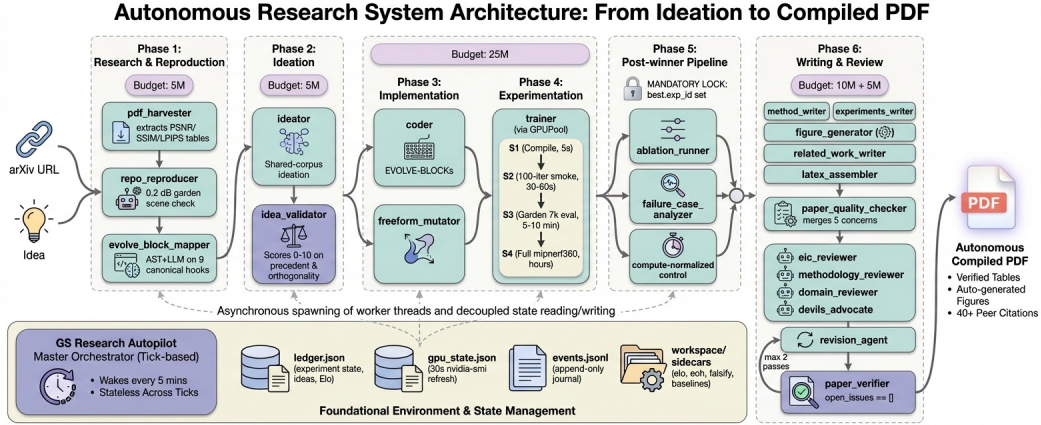


Figure 1: IDEA2PAPER architecture. A prompt or a target-paper URL enters at the left. Stage 1 reproduces the target on one scene and measures a noise floor before any modification is allowed. Stage 2 generates literature-grounded hypotheses over a taxonomy of seven 3DGS novelty categories, filtered by a cheap idea validator and by category-level saturation statistics. Stage 3 wraps the regions of code that belong to each category with EVOLVE-BLOCK markers, using a cross-codebase mapper so the scaffold transfers to any SOTA 3DGS repository. Stage 4 runs a hybrid evolutionary loop that unifies EoH rationale-implementation pairs, a grounded Elo ladder driven by real ns-train PSNR, and ReEvo-style verbal gradients restricted to surprising pairs and partitioned by category. Stage 5 filters candidates through a four-stage compute cascade and three adversarial attacks whose outcomes subtract Elo, making robustness a first-class fitness signal. Stage 6 governs the whole run as a three-level academic hierarchy: a master *professor* agent (a heavy-reasoning model) decides each step, PhD-student *phase coordinators* manage sub-workflows, and undergraduate *leaf agents* execute atomic tasks. A two-tier context-compaction scheme keeps the professor on task across long runs by pinning a 10k-token summary of the run’s early history and only summarising new action deltas thereafter. Stage 7 writes the paper with VLM figure verification and a claim-backed verifier that gates termination. Every reported number in the output paper is forced by construction to trace back to a measurement the leaf agents actually produced.

1 Introduction

2 Related Work

3 Method

3.1 Overview and Design Principles

GS-SCIENTIST turns a one-line research prompt or a target-paper URL into a publication-ready 3D Gaussian Splatting (3DGS) paper (?) with real GPU-measured PSNR, SSIM, and LPIPS. The system orchestrates 20+ specialised agents across seven stages under a three-level academic hierarchy, and is built around four design principles that together distinguish it from prior autonomous-research systems (????????).

P1. Hierarchical agent orchestration. The run is driven by a single *master orchestrator* agent (a heavy-reasoning model; concrete model IDs are reported in the experiments section) that plays the role of a principal investigator: it decides at every step which sub-workflow to run next or which leaf agent to spawn. Beneath the master, six *phase coordinators* play the role of PhD students: each knows how to manage its own stage (grounding, ideation, evolution, refinement, writing, review) end-to-end. Beneath the coordinators, the *leaf agents* play the role of undergraduates: each does one tightly-scoped atomic task (clone a repo, run ns-train, apply a code patch, draft a LaTeX section, score an idea, run an adversarial attack). A professor does not write code, a PhD does not train GPUs, and an undergrad does not re-plan the research agenda: each level owns its own decisions. In contrast

Table 1: The seven stages of IDEA2PAPER. Each phase is managed end-to-end by a PhD-student phase coordinator.

Stage	Input	Output
1. Grounding	prompt or paper URL	reproduced target, noise floor, peer baselines
2. Hypothesis generation	literature corpus, target repo	validated idea queue with category tags
3. Code region isolation	cloned repo, idea categories	EVOLVE-BLOCK-annotated source tree
4. Evolutionary search	annotated tree	Elo-ranked candidate pool with rationales
5. Cascade + falsification	candidate pool	winner, ablation, failure cases
6. Master orchestration	run state across phases	per-step decisions, compaction passes
7. Writing + peer review	winner + measurements	verified LaTeX paper + PDF

to AI Scientist’s 300-turn single-session master (?) whose context fills before a paper completes, the professor invokes short bounded sessions that delegate through the hierarchy.

P2. Context compaction keeps the professor on task. A 24-to-48 hour autonomous run produces an action log that would overflow any context window. We compact the master’s memory in two tiers. When the professor’s estimated input exceeds 500k tokens, a cheap summariser agent distills the entire run so far into a 10k-token *pinned* summary that captures the target repo, reproduced baseline PSNR and training time, discovered entryptoint framework, noise-floor standard deviation, per-category trials/wins/saturation, best experiment id, structural failures, and any open paper issues. This pinned summary is *never* rewritten for the rest of the run. Every subsequent compaction only produces a 4k-token *delta* summary of actions since the last compaction, appended after the pinned block. The last eight raw actions are kept verbatim. The professor therefore always sees the anchor facts of the run first, then a handful of delta summaries, then the most recent actions, which prevents goal drift: the pinned summary says “your job is to beat target X on metric Y and no paper has shipped yet” and survives every context reset.

P3. Grounded evaluation, grounded claims. A candidate’s fitness is always a real ns-train PSNR measurement against a cached splatfacto baseline, never a model-reported or language-model-judged score. A paper’s numerical claims must resolve to a backing ledger entry before the run can terminate as successful. These two rules close the two failure modes that the AI Scientist post-mortem, AgentRxiv’s self-reported evaluation, and CycleResearcher’s purely textual generation each exhibit (??): methods that only look good on self-reported metrics, and papers whose numbers were never actually measured.

P4. Falsification inside the loop. Every candidate that passes the cheap compute cascade is subjected to three adversarial attacks whose outcomes subtract Elo. No prior autonomous-research pipeline makes robustness a fitness signal; reported results are taken at face value. We make the search itself prefer methods that survive stress tests.

The seven stages (Figure ??) execute in a fixed topological order but with phase-internal iteration and re-entry. Table ?? summarises their inputs and outputs. The pipeline optionally accepts a per-run *lab budget* (USD) that a lab administrator can configure; when set, it is distributed proportionally across phases and enforced by the SDK at every agent invocation.

3.2 Stage 1: Grounding the Problem

The first stage closes the most common failure mode of LLM-written science: writing about numbers the system never actually measured. Before any candidate method is generated, the pipeline binds itself to a concrete, reproducible baseline.

Structure-preserving PDF ingestion. A *pdf harvester* agent invokes MinerU (?) to convert the target PDF into markdown that preserves equations, tables, and figure captions. Prior autonomous-research pipelines (??) typically fall back on plaintext extraction, which silently loses table structure and forces downstream agents to re-read the PDF. Our harvester parses every results table into a dictionary of (method, scene, metric) triples, yielding two artefacts: the target’s own reported numbers, and every peer baseline row from the same table as *free* comparison data. Only the target

itself is reproduced on our GPUs; the rest of the comparison table is harvested once and never retrained. This cuts baseline-compute cost from $O(N_{\text{methods}})$ GPU-weeks to $O(1)$.

Reproduce-first contract. A *repo_reproducer* agent clones the target’s git repository and trains one representative scene (garden at 7,000 iterations) using nerfstudio (??) as the execution DSL if needed. If the reproduced PSNR is within 0.2 dB of the paper’s reported value, the run is authorised to proceed; otherwise *no* experiments are permitted to spawn and a debugger agent is invoked to diagnose the discrepancy (wrong eval crop, missing test-time augmentation, config drift). This contract is what keeps the experiment budget bounded in practice: most prior pipelines (??) begin modifying before confirming they can match the baseline, which invalidates every downstream comparison.

Noise-floor protocol. The baseline is then retrained with $N \geq 3$ seeds on the representative scene, and the per-scene standard deviation σ is recorded in `ledger.noise_floor`. A candidate is allowed to be declared a winner only if its PSNR delta Δ against the baseline satisfies

$$\Delta \geq 2\sigma, \quad (1)$$

with a conservative $\sigma = 0.1$ dB seeded at run start and overwritten by the empirical measurement. This protocol, absent from every prior autonomous-research baseline we surveyed, directly addresses the “+0.08 dB counts as a contribution” false-positive trap.

3.3 Stage 2: Hypothesis Generation over a Novelty Taxonomy

Rather than ask the ideator to generate ideas over an unbounded space, we give it a structured taxonomy of seven 3DGS novelty categories. Each category denotes a well-defined locus of modification and has a clear precedent in the recent literature, so the ideator’s output is both tractable and grounded.

The seven categories. Let $\mathcal{C} = \{C_1, \dots, C_7\}$ denote the taxonomy. We define it as:

- C_1 **Loss and regularisation.** New reconstruction losses, perceptual losses, or regularisers added to the optimisation objective. Prior work: FreGS frequency loss (?), depth-regularised 3DGS (?), opacity or scale regularisers.
- C_2 **Densification and pruning.** Rules that add, clone, split, or remove Gaussians during training. Prior work: AbsGS (?), Pixel-GS (?), revised splitting (?), MCMC-3DGS (?), efficient density control (?), generative densification (?).
- C_3 **Rendering and rasterisation.** Forward-pass changes including anti-aliasing, projection, and alpha-blending. Prior work: Mip-Splatting (?), 2DGS (?), analytic splatting (?), StopThePop (?), multi-scale 3DGS (?).
- C_4 **Parameter representation.** Covariance parameterisation, colour representation, and activation choices. Prior work: GES generalised exponentials (?), effective rank (?), spec-Gaussian (?), Gaussian Shader (?).
- C_5 **Optimisation and scheduling.** Optimiser choice, learning-rate schedules, curricula, and training-time speedups. Prior work: Taming-3DGS (?), DashGaussian (?), FasterGS (?), TrickGS (?), SteepGS second-order optimisation (?).
- C_6 **Scaffold and structural priors.** Anchor-point or hierarchical structures layered on top of raw Gaussians. Prior work: Scaffold-GS (?), Octree-GS (?), Fast-GS (?).
- C_7 **Geometric and view-sparse priors.** Depth or normal priors and few-view training. Prior work: FSGS (?), DNGaussian (?), CoR-GS (?), DepthSplat (?), depth-regularised 3DGS (?).

The taxonomy is the unified interface between ideation (Stage 2), code isolation (Stage 3), bandit-style credit assignment (Stage 4), and verbal-gradient routing (Stage 4). It is the palette the agent uses to *improve* the target or to *propose* a new method.

Literature-grounded ideator. The *ideator* agent operates under a strict provenance rule: every proposed idea carries a category tag $C_k \in \mathcal{C}$, a short rationale, and an `inspired_by` field populated with real bibliography keys. Ideas of the form “technique Y was shown to help NeRF (?) in category

141 C_k , so we adapt Y to 3DGS via a specific modification m_Y ” are preferred over ungrounded intuitions.
 142 Because `inspired_by` is mechanically traced into the related-work writer’s citation set, the story
 143 told at ideation time and the story told in the paper are forced to agree. This rule replaces the free-form
 144 “it feels like it would work” ideation exhibited by AI Scientist’s archive-expansion loop (?).

145 **Idea validator as a cheap pre-gate.** Training a 3DGS candidate on one scene at 7,000 iterations
 146 costs five to ten minutes of GPU. Scoring an idea with a short heavy-model call costs cents. Before
 147 any training happens, an *idea_validator* scores each queued idea on four criteria: expected gain from
 148 literature precedent, availability of the target category in the cross-repo hook mapping, category
 149 saturation (see next paragraph), and orthogonality to the current best. Ensemble or seed-averaging
 150 tricks auto-floor at 6/10; bare learning-rate tweaks cap at 3/10. Ideas below threshold are rejected
 151 before they ever consume GPU time.

152 **Category-level bandit.** The ledger maintains a table `category_stats`: $\mathcal{C} \rightarrow (t, w, \bar{\Delta}, \Delta^*)$ where,
 153 for each category, t is the number of trials, w is the number of wins, $\bar{\Delta}$ is the running mean PSNR
 154 delta, and Δ^* is the best delta. When $t \geq 5$ and $w = 0$, the category is marked *saturated*, and
 155 the ideator is forbidden from proposing further ideas under that tag for the remainder of the run.
 156 This replaces the pattern we observed in early full runs, where 27 consecutive loss-function tweaks
 157 produced zero wins before the system was redirected to densification-side modifications. Agent
 158 Laboratory, MLR-Copilot, and the AI Scientist line have no comparable mechanism; their ideators
 159 either iterate inside the same category until budget runs out or blindly accept diversity heuristics.

160 3.4 Stage 3: Code Region Isolation via EVOLVE-BLOCK Markers

161 A category tag is semantic. To mutate code safely we need a syntactic scope. Stage 3 injects
 162 comment-based markers that bind each category to the regions of source that implement it.

163 **Marker protocol.** The annotator wraps each region of the model source that corresponds to a
 164 category with a pair

```
165 # EVOLVE-BLOCK-START <region_name>
166 ...body...
167 # EVOLVE-BLOCK-END <region_name>
```

168 so that a diff can be mechanically scoped to one region. A mutator agent invoked with a set of region
 169 names is instructed by its prompt that any diff touching code outside those marker pairs will be
 170 rejected before the patch is applied. The annotator is idempotent: re-annotating a partially marked
 171 file is a no-op. Two concurrent mutations against disjoint region sets therefore cannot clobber each
 172 other, which is what permits the parallel generational structure of Stage 4. FunSearch mutates at
 173 function granularity (?), which is too narrow for methods that span rendering, densification, and loss;
 174 AlphaEvolve (?) generalises to arbitrary marked regions but leaves their placement to the user. We
 175 commit to a category-aligned placement.

176 **Cross-codebase hook mapping.** The region vocabulary is defined against a reference 3DGS
 177 template, but most SOTA 3DGS repositories (Mip-Splatting, Scaffold-GS, 2DGS, GES) use their
 178 own file layouts. An *evolve_block_mapper* agent combines AST analysis with LLM reasoning to
 179 identify the analogue of each category’s region in any cloned repository, injecting LLM markers in-place
 180 and reporting `{regions_found, confidence, mapping}` to the orchestrator. The orchestrator
 181 then picks a track: if at least five of the seven category regions are recovered, it runs Track A, where
 182 every downstream mechanism (EoH, Elo, falsification, category-partitioned verbal gradients) applies
 183 unchanged; otherwise it falls back to Track B, a *freeform_mutator* that emits unified diffs against
 184 arbitrary files using the same rationale-pair and Elo machinery but without per-category lesson
 185 grouping. This graceful degradation lets our evolutionary infrastructure transfer to any 3DGS-shaped
 186 codebase without per-repo human annotation.

187 3.5 Stage 4: Hybrid Evolutionary Search

188 Candidates compete inside one evolutionary loop that unifies three ideas from the program-search
 189 literature and adds credit assignment none of them uses.

Rationale-implementation pairs and operators. Following EoH (?), every candidate is a tuple (rationale, impl_path, parents, categories, r , PSNR), where the rationale is a short (≤ 25 -word) natural-language statement the mutator emits alongside the code. Five prompt-specialised operators are scheduled ε -greedy ($\varepsilon=0.25$) over per-operator average Elo-delta gain: two crossovers (conservative, exploratory), a single-parent refine, a numeric-only tweak, and a structural rewrite. The first slot of every generation is forced to the conservative refine. Parents are drawn from a softmax over Elo ratings with $\tau=200$; arity-two operators replace one parent with a uniform draw, guaranteeing diversity. Per-operator Elo-delta credit assignment is absent from EoH (fixed schedule) and FunSearch (?) (no distinct operators).

Grounded Elo with margin scaling and falsification coupling. Within each generation we run an all-pairs tournament; the higher-PSNR candidate wins. Ratings update with the standard Elo formula using $K=32$ and a margin-aware scale $s(m_{dB}) = \text{clamp}(0.75 + 0.25 m_{dB}, 0.5, 1.5)$, so a two-decibel blowout moves ratings $1.5 \times K$ and a photo finish $\approx 0.78 \times K$. Beyond the pairwise delta, each Stage 5 attack a candidate fails subtracts 30 Elo, making robustness a direct selection signal: a method that wins by 0.8 dB but fails the ablation attack can lose rank to one that won by 0.4 dB and survived. Crucially, the PSNRs driving the ladder are real ns-train measurements on a cached splatfacto baseline, not language-model-judged scores, unlike the text-only Elo used by AI Co-Scientist (?).

Verbal gradients, upset-restricted and per-category. After Elo updates we identify *surprising* pairs — those where the loser’s pre-match rating was within 60 Elo of the winner’s. For up to four such upsets per generation, a cheap summariser receives the two candidates’ diffs, rationales, and PSNR delta and writes a short lesson tagged by category. On the next generation, any operator touching category C_k sees only the lessons tagged C_k . ReEvo (?) runs reflective gradients on every pair ($O(n^2)$); our upset-and-category restriction reduces this to at most four cheap calls per generation.

Re-ideation on plateau. After three generations, if no candidate beats the baseline by 2σ , a *negative_ideator* heavy-model agent reads every failed rationale, the per-category saturation table, and the baseline architecture, and is instructed to emit a seed that is *structurally different* from the failed directions. Its output is re-annotated and fed into a fresh cycle; Elo state and the operator bandit persist across cycles. We cap at two re-ideations (three cycles total). FunSearch, AlphaEvolve, and EoH all assume the seed is good enough; we detect plateau and respawn at a higher level of abstraction.

3.6 Stage 5: Evaluation Cascade and Adversarial Falsification

Stage 5 decides which candidates influence selection: a cost-aware cascade (extending AlphaEvolve (?)) plus an adversarial robustness pass with no analogue in the autonomous-research literature.

Four-stage cascade. Every candidate is pushed through:

- **S1 (compile check):** Python import and syntactic sanity, ~ 5 s, no GPU.
- **S2 (smoke train):** 100 iterations on garden, 30–60 s on one GPU. Must complete with finite loss.
- **S3 (single-scene short eval):** 7,000 iterations on the representative scene, 5–10 min on one GPU. Must satisfy $\text{psnr} \geq \text{baseline}_{\text{splatfacto}} - 1.0$ dB, with the baseline cached per (scene, stage) and reused across candidates.
- **S4 (full benchmark):** 30,000 iterations on the full Mip-NeRF 360 (?) nine-scene suite, hours per scene. Runs only on the single tournament champion.

A GPU pool enforces a one-candidate-one-GPU invariant: one candidate holds one device for the entirety of its cascade run, candidates run concurrently on different devices, and each generation is synchronised between stages. S4’s scenes run sequentially on the champion’s single device. Without the cascade, a three-generation, eight-island run on Mip-NeRF 360 would cost upward of 150 GPU-days; with it, 23 of 24 candidates are filtered out at S3 and only the champion pays S4.

239 **Three adversarial attacks.** Every candidate that clears S3 is subjected to three attacks before it is
 240 allowed to update the Elo ladder for the next round.

- 241 • **A1 Edge scene.** Train on the harder scene `flowers` (low `splatfacto` baseline, known to
 242 break methods overfit to dense-structure scenes) for 3,000 iterations. Pass iff `psnr` $\geq$
 243 `baselineflowers - 2.0 dB`.
- 244 • **A2 Sparse view.** Re-run the representative scene with only three training views. Pass iff the
 245 full-to-sparse PSNR gap does not exceed the baseline sparse gap by more than 2.0 dB.
- 246 • **A3 Hyper-parameter ablation with load-bearing semantics.** Regex-extract the first
 247 numeric hyper-parameter named in the rationale (e.g., “ $\gamma = 0.35$ ”), zero it in-place with
 248 a `.falsify-bak` backup, and re-run S3. Pass iff either (i) removing the hyper-parameter
 249 costs at least 0.3 dB, or (ii) removing it crashes training. The second condition is important:
 250 a crash means the hyper-parameter is load-bearing, which is exactly what we want to verify.
 251 A method whose “contribution” is a no-op fails this test.

252 The falsification score combines the representative-scene PSNR with weighted penalties,

$$f = w_{\text{garden}} \text{psnr}_{\text{garden}} - 1.5 \mathbb{1}[A1 \text{ fails}] - 1.0 \mathbb{1}[A2 \text{ fails}] - 2.0 \mathbb{1}[A3 \text{ fails}], \quad (2)$$

253 with $w_{\text{garden}} = 1.0$; the ablation penalty is largest because over-claiming is the worst failure mode
 254 for a research method. Each failure also subtracts 30 Elo, so falsification steers evolutionary selection
 255 directly. Red-teaming for language-model outputs exists (?), but no prior autonomous-research
 256 pipeline runs an adversarial pass on its generated methods inside the optimisation loop.

257 **Compute-normalised controls.** Once the champion is locked, the pipeline spawns a compute-
 258 normalised control that gives the same extra compute (seeds, iterations, ensemble size) to the baseline,
 259 an ablation runner that removes each claimed mechanism one at a time, and a failure-case analyser
 260 that renders failure crops for the qualitative figure.

261 3.7 Stage 6: Hierarchical Agent Orchestration with Context Compaction

262 Stages 1–5 describe *what* the system does; Stage 6 governs *who decides*. The design is borrowed
 263 from academia: one professor, several PhD students, and a pool of undergraduates, each with its own
 264 scope and context.

265 **Three-level hierarchy.** A single heavy-reasoning *professor* agent decides, per step, which sub-
 266 workflow to run or which leaf agent to spawn; it never writes code, trains on GPUs, or drafts LaTeX.
 267 Six *PhD-student* coordinators own the sequencing of their own phase (grounding, ideation, evolution,
 268 refinement, writing, review); the evolution coordinator, for instance, plans operators, runs the Elo
 269 tournament, and triggers verbal-gradient calls. 20+ *undergraduate* leaf agents each execute one
 270 atomic task (researcher, reproducer, coder, trainer, falsifier, idea validator, negative-ideator, method
 271 writer, section writer, figure generator, teaser composer, qualitative-grid composer, LaTeX assembler,
 272 paper verifier, reviewer roles). A leaf knows only its task; a PhD knows only its phase; the professor
 273 sees the run. Dispatch cost is confined to the professor, action cost to the leaves, and no single leaf
 274 failure can derail the run.

275 **Per-step decision.** At each step the professor’s user prompt concatenates a pinned summary, delta
 276 summaries, the last eight raw actions, a minimal run-state snapshot, and a budget snapshot. It emits
 277 exactly one JSON decision: `run_phase` (delegate a whole phase), `spawn_agent` (surgical invocation
 278 of one leaf with structured args — used for retrying a failed trainer or re-running a single writer
 279 when the best experiment changes), or `stop`. Stopping is only permitted when all four completion
 280 preconditions hold: target beaten on every must-beat metric, every numerical claim cleared by the
 281 verifier, a compiled paper `.pdf` on disk, and the draft not stale with respect to the current best.

282 **Two-tier context compaction.** Let $T(k)$ be the professor’s estimated prompt size at step k and
 283 $\mathcal{A}_k$ its raw action log. When $T(k) > 500\text{k}$ tokens a cheap summariser runs. The *first* compaction
 284 produces a pinned summary Σ_0 of $\leq 10\text{k}$ tokens that preserves a prescribed checklist (target repo,
 285 reproduced PSNR and training time, entrypoint framework, noise-floor σ , per-category (*tried*, *wins*),
 286 current best `.exp_id` and delta, structural failures, open paper issues). Σ_0 is *never rewritten*. Every

subsequent compaction emits only a $\leq 4k$ -token delta Σ_i covering actions since the last compaction; the summariser is explicitly prompted that earlier summaries must not be reprocessed. The last eight raw actions survive verbatim. The professor’s memory at step k is thus $\mathcal{M}_k = \Sigma_0 \parallel \Sigma_1 \parallel \dots \parallel \Sigma_{n_k} \parallel \mathcal{A}_k^{(last-8)}$, with Σ_0 always first. Because Σ_0 is immovable, the research goal and outstanding issues are present on every single step, which in practice prevents the long-horizon drift exhibited by context-free agentic loops (?).

Best-first experiment tree. Within the evolution coordinator, experiments form a tree; each node is an idea and children refine their parent, following AI Scientist-v2 (?). We score a node n as

$$s(n) = 1.0 \Delta_{smoke}(n) + 0.6 \nu(n) - 0.15 d(n) - 0.02 c(n) - 10 \mathbb{1}[debug(n) \geq 2], \quad (3)$$

with Δ_{smoke} the 500-iter PSNR differential, ν validator novelty, d depth, c cumulative compute spend, and the last term hard-killing nodes that failed twice in a row. Expansion is priority-queue based; when the frontier maximum falls below -2.0 the phase halts. Δ_{smoke} is a real ns-train measurement, so the score cannot be gamed by the agent — unlike AI Scientist-v2’s self-reported leaf metric.

Optional lab budget. An administrator may set a single per-run budget; when provided it is distributed proportionally across phases and SDK-enforced at every agent invocation. Otherwise runs are bounded only by the completion predicate, per-agent turn caps, and a wall-clock limit.

Shared baseline library. A cross-run library at `$IDEA2PAPER_LIBRARY_ROOT` (default `~/.idea2paper/library/`) is symlinked into every workspace as `literature/shared_library/`. It holds one ~ 1500 -token summary per paper plus the full MineRU (?) extraction, keyed by bibkey. The researcher reads this library first, classifies summaries into the seven novelty categories, and only fetches 5–10 new papers to close the thinnest gaps; new acquisitions are swept back into the library at phase end. This converts literature discovery from an $O(40)$ MineRU crawl into an $O(1)$ disk read plus a bounded top-up.

3.8 Stage 7: Grounded Writing, VLM Verification, and Simulated Peer Review

The final stage turns accumulated measurements into a paper under two hard preconditions: incrementality (so text converges rather than drifts) and claim-backed verification (so the paper cannot ship numbers that were never measured).

Incremental writers. Six section writers (abstract, introduction, related work, method, experiments, conclusion) each read the existing section, diff numbers against the current best, preserve untouched prose, and converge across successive calls. The related-work writer is bound by a citation floor of 40 verified bib entries. If `best.exp_id` has changed since `paper.last_written_at`, the draft is stale and the professor must spawn a revision before stopping.

VLM figure verification. A `figure_vlm_checker` opens every figure and verifies that axis labels match the caption, legend entries match the paper’s method names, subfigure ordering matches ($a/b/c$), and numbers visible in the figure match the measurements. AI Scientist-v2’s VLM loop (?) refines figure aesthetics; ours audits figure *correctness*.

Claim verification as a stop-condition gate. A `paper_verifier` scans the compiled draft for every numerical claim and searches for a backing measurement. Unbacked claims become open issues; the run cannot terminate while any remain. The professor’s next step then either spawns experiments to back a claim, a revision agent to weaken it, or a negative ideator if the claim signals a structural failure. This turns claim integrity from a prompt-level guideline into a runtime gate, unlike CycleResearcher’s purely textual generation (?) and AgentRxiv’s self-reported shared-preprint loop (?).

Simulated peer review. Four reviewers run in parallel (editor-in-chief, methodology, domain, devil’s advocate); an editorial synthesiser merges their reports and a revision agent edits the draft. The review-revise-check loop runs up to two passes by default. CycleResearcher uses review as an RL signal on writing; we use it as a final integrity gate with explicit contradiction resolution rather than score averaging.

4 Experiments

4.1 Experimental Setup

4.2 Quality of Generated 3DGS Methods

4.2.1 End-to-End Success Rate

4.2.2 Quantitative Comparison vs. State of the Art

Across twelve runs (2026-04-18 to 2026-05-01) with different SOTA targets named at run-start, three winners cleared the noise-floor gate ($\Delta \geq 2\sigma$, §??) and improved on the reproduced target on the run’s headline metric. Table ?? lists them. Each *SOTA* row is the target reproduced on our GPUs under the reproduce-first contract, not paraphrased from the target paper; the *limitation* column is the bottleneck-analyst diagnosis the ideator was conditioned on for that run.

Table 2: Reproduced SOTA vs. IDEA2PAPER output. Top block: completed runs whose winner beat its reproduced SOTA on the run’s headline metric. Bottom block: compositional discovery loop, which reproduces a recent paper and then composes additive modifications from the Stage-2 taxonomy. “Things added / fix” is the bottleneck-analyst diagnosis (top) or the validated mechanism stack (bottom). Bold marks the better number in each pair. FreezeGS is reported at iso-wall (109 s) where SSIM/LPIPS were not logged for the matched FastGS budget. “#GS” is the final Gaussian count in millions; entries left blank were not logged.

Setting	Method	Limitation attacked / fix	PSNR $\uparrow$	SSIM $\uparrow$	LPIPS $\downarrow$	#GS
<i>Run winners (target named at run-start)</i>						
Mip-NeRF 360	FastGS (?)	{0, 1} thresholded pixel-count is frequency-blind; edges and shading errors rank identically	29.12	0.858	0.217	—
7 sc., 30k it.	TileFocalGS (ours)	rank-graded per-tile weight, EMA across views	29.69	0.869	0.194	—
Mip-NeRF 360	FastGS (?)	opacity reset every 3K iters destabilises late refinement; re-set keeps firing after densification dies	25.65	—	—	—
9 sc., iso-wall 109 s	FreezeGS (ours)	freeze geometry + drop reset in phase 2	26.08	—	—	—
Nebula	splatfacto (?)	pinhole rasteriser ignores OPENCV distortion; SfM triangulates ≈ 0 stars (sub-pixel sources)	32.58	0.914	0.278	0.07
180 fr. 1080p, 30k it.	Nebula3DGS-v2 (ours)	3DGS-MCMC (?) sampler + saturation-driven star seeder	36.36	0.945	0.201	—
<i>Compositional discovery (reproduce + extend; same scene as the target paper)</i>						
Set 1	Opti3DGS ? + discovery (ours)	reproduction-only baseline adaptive densif. threshold + opacity entropy reg.	27.71 28.05	0.841 0.853	0.209 0.125	1.40 4.95
Set 1	MSAA 3DGS ? + discovery (ours)	reproduction-only baseline bilateral grid + extended training (+15K iters)	25.35 26.19	0.758 0.791	0.369 0.293	2.73 2.96
Set 2	GNS ? + discovery (ours)	reproduction-only baseline AA rasterisation + denser init + post-densif. losses	25.97 26.15	0.891 0.902	0.095 0.097	0.49 0.61
Set 2	DWTGS ? + discovery (ours)	reproduction-only baseline patch structure loss + opacity/normal/depth reg.	18.68 21.09	0.510 0.725	0.374 0.281	1.64 2.43
Set 2	CompGS ? + discovery (ours)	reproduction-only baseline EMA centroid updates + grad-based codebook refine	28.28 28.50	0.894 0.899	0.153 0.146	0.67 0.63

For the run winners, the winning method targets an EVOLVE-BLOCK category the analyst had flagged as under-attacked before ideation began. TileFocalGS attacks C_1 (loss/regularisation): a

per-tile rank-graded weight in place of the $\{0, 1\}$ count, the LPIPS drop of 0.022 matching the gap the analyst predicted. FreezeGS attacks C_5 (optimisation/scheduling). Nebula3DGS-v2 attacks C_2 (densification/pruning), with the bulk of the +3.78 dB concentrated on star-region recovery rather than the diffuse gas. For the compositional-discovery block, the largest gain is DWTGS (+2.41 dB PSNR, +0.215 SSIM) and the most efficient is CompGS, which improves on every metric while reducing the Gaussian count by 6%. The remaining nine non-discovery runs did not clear the gate; per the claim-backed verification gate (§??), each terminated as a research log rather than a paper.

4.2.3 Qualitative Renders and Visual Comparisons

4.3 Per-Stage Evaluation

4.3.1 Phase 1: Research and Literature Grounding

4.3.2 Phase 2: Evaluation of Generated Research Ideas

A natural critique of any automated research system is that the “novelty” it produces is in the eye of the LLM, not the field. We address this by adapting the blind expert review protocol of ? to 3DGS and pairing it with an execution check.

Study design. We collect 30 idea proposals: 20 from our pipeline (Phase 2 ideation, before execution-based refinement) and 10 written by human researchers, all following a fixed template (problem, motivation, method, experiment plan, fallback). All 30 are passed through an LLM-based style normalizer that rewrites them into a uniform voice without altering content, controlling for authorship cues (?). Then we asked 10 expert reviewers, each with at least one peer-reviewed publication in neural rendering. Reviewers were blind to the source of each idea. The 10 human ideas were written by PhD students in the same area. Each idea was reviewed by all 10 reviewers, yielding 300 reviews total. We use the four breakdown metrics from ? on a 1-10 scale: novelty, excitement, feasibility, expected effectiveness, plus an overall score. Each score is paired with a free-text rationale, and anchor descriptions are provided at each scale point. The full form is in Appendix ??.

AI ideas score higher on novelty. Table ?? reports mean review scores per condition. AI ideas receive a higher mean novelty score than human ideas (vs). AI ideas score lower on feasibility (vs), though this gap is not significant. Excitement, expected effectiveness, and overall score all favor AI by smaller, non-significant margins.

Running the remaining pipeline phases on all 20 AI ideas, 13 produced methods that trained to convergence on Mip-NeRF 360. Mean PSNR was **27.55** dB versus 27.41 dB for 3DGS (?) under matched training budget.

377	4.3.3 Phase 3: Implementation and Coder Agent
378	4.3.4 Phase 4: Experimentation
379	4.3.5 Phase 5: Analysis, Ablations, and Failure Cases
380	4.3.6 Phase 6: Paper Writing
381	4.3.7 Phase 7: Peer Review Simulation
382	4.3.8 Cross-Stage Failure Localization
383	4.4 LLM-as-Judge Evaluation
384	4.4.1 Multi-Model Ensemble Protocol
385	4.4.2 Position-Swap and Self-Enhancement Calibration
386	4.4.3 Human–LLM Agreement
387	4.4.4 Bias Audit
388	4.5 Blind Human Expert Study
389	4.5.1 Ideation Arm
390	4.5.2 Paper Arm: Paired AI vs. Human Comparison
391	4.5.3 Inter-Rater Reliability and Statistical Analysis
392	4.6 Real Workshop Submission
393	4.6.1 Submission Protocol and Ethical Prerequisites
394	4.6.2 Acceptance Outcomes and Reviewer Feedback Analysis
395	4.7 Ablation Studies
396	4.7.1 Phase Removal
397	4.7.2 Agent Count
398	4.7.3 Evolutionary Tournament Population
399	4.7.4 Dynamic Orchestration vs. Static DAG
400	4.8 Cost and Wall-Clock Analysis
401	4.8.1 API, GPU, and Time Accounting
402	4.8.2 Quality–Cost Pareto Frontier
403	5 Conclusion

404 **References**

- 405 Anil Armagan, Albert Saà-Garriga, Benedetto Manganelli, Marcin Nowak, and Mehmet Kerim
406 Yucel. Trick-GS: A balanced bag of tricks for efficient Gaussian Splatting. In *IEEE International*
407 *Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.
- 408 Jonathan T Barron, Ben Mildenhall, Dor Verbin, Pratul P Srinivasan, and Peter Hedman. Mip-NeRF
409 360: Unbounded anti-aliased neural radiance fields. In *IEEE/CVF Conference on Computer Vision*
410 *and Pattern Recognition (CVPR)*, pages 5470–5479, 2022.
- 411 Samuel Rota Bulò, Lorenzo Porzi, and Peter Kotschieder. Revising densification in Gaussian
412 Splatting. In *European Conference on Computer Vision (ECCV)*, 2024.

413 Youyu Chen, Junjun Jiang, Kui Jiang, Xiao Tang, Zhihao Li, Xianming Liu, and Yinyu Nie. Dash-
414 Gaussian: Optimizing 3D Gaussian Splatting in 200 seconds. In *IEEE/CVF Conference on*
415 *Computer Vision and Pattern Recognition (CVPR)*, pages 11146–11155, 2025.

416 Jaeyoung Chung, Jeongtaek Oh, and Kyoung Mu Lee. Depth-regularized optimization for 3D
417 Gaussian Splatting in few-shot images. In *CVPR Workshop on 3D Vision and Modeling*, 2024.

418 Xiaobin Deng, Changyu Diao, Min Li, Ruohan Yu, and Duanqing Xu. Efficient density control for
419 3D Gaussian Splatting, 2024.

420 Juraj Gottweis, Wei-Hung Weng, Alexander Daryin, Tao Tu, Anil Palepu, Petar Sirkovic, Artiom
421 Myaskovsky, Felix Weissenberger, Keran Rong, Ryutaro Tanno, Khaled Saab, Dan Popovici,
422 Jacob Blum, Fan Zhang, Katherine Chou, Avinatan Hassidim, Burak Gokturk, Amirata Vahdat,
423 Pushmeet Kohli, Yossi Matias, Andrew Carroll, Kavita Kulkarni, Nenad Tomasev, Yuan Guan,
424 Vikram Dhillon, Eeshit Dhaval Vaishnav, Byron Lee, Tiago R. D. Costa, Jose R. Penadés, Gary
425 Peltz, Yunhan Xu, Annalisa Pawlosky, Alan Karthikesalingam, and Vivek Natarajan. Towards an
426 AI co-scientist. *arXiv preprint arXiv:2502.18864*, 2025.

427 Florian Hahlbohm, Linus Franke, Martin Eisemann, and Marcus Magnor. Faster-GS: Analyzing and
428 improving Gaussian Splatting optimization. In *IEEE/CVF Conference on Computer Vision and*
429 *Pattern Recognition (CVPR)*, 2026.

430 Abdullah Hamdi, Luke Murai, Jinjie Shi, Florian Bernard, Matthias Nießner, Andrew J. Davison, and
431 Federico Tombari. GES: Generalized exponential splatting for efficient radiance field rendering.
432 In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

433 Binbin Huang, Zehao Yu, Anpei Chen, Andreas Geiger, and Shenghua Gao. 2D Gaussian Splatting
434 for geometrically accurate radiance fields. *ACM Transactions on Graphics (SIGGRAPH)*, 43(4),
435 2024.

436 Junha Hyung, Susung Han, Sungwon Kim, Jaeseong Hong, and Jaegul Choi. Effective rank analysis
437 and regularization for enhanced 3D Gaussian Splatting. In *European Conference on Computer*
438 *Vision (ECCV)*, 2024.

439 Yingwenqi Jiang, Jiadong Tu, Yuan Liu, Xifeng Gao, Xiaoxiao Long, Wenping Wang, and Yuexin
440 Ma. GaussianShader: 3D Gaussian Splatting with shading functions for reflective surfaces. In
441 *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

442 Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian
443 splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023.
444 doi: 10.1145/3592433.

445 Shakiba Kheradmand, Daniel Rebain, Gopal Sharma, Weiwei Sun, Jeff Tseng, Hossam Isack,
446 Abhishek Kar, Andrea Tagliasacchi, and Kwang Moo Yi. 3D Gaussian Splatting as Markov Chain
447 Monte Carlo. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

448 Jiahe Li, Jiawei Zhang, Xiao Bai, Jin Zheng, Xin Ning, Jun Zhou, and Lin Gu. DNGaussian:
449 Optimizing sparse-view 3D Gaussian radiance fields with global-local depth normalization. In
450 *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.

451 Ruochen Li, Teerth Patel, Qingyun Wang, Qingyun Wang, and Xinya Du. MLR-Copilot: Autonomous
452 machine learning research based on large language model agents. In *NAACL*, 2025.

453 Zhihao Liang, Qi Zhang, Ying Feng, Ying Shan, and Kui Jia. Analytic-Splatting: Anti-aliased 3D
454 Gaussian Splatting via analytic integration. In *European Conference on Computer Vision (ECCV)*,
455 2024. Oral.

456 Fei Liu, Xialiang Tong, Mingxuan Yuan, Xi Lin, Fu Luo, Zhenkun Wang, Zhichao Lu, and Qingfu
457 Zhang. Evolution of heuristics: Towards efficient automatic algorithm design using large language
458 models. In *International Conference on Machine Learning (ICML)*, 2024.

459 Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. The AI Scientist:
460 Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*, 2024a.

461 Tao Lu, Mulin Yu, Linning Xu, Yuanbo Xiangli, Limin Wang, Dahua Lin, and Bo Dai. Scaffold-GS:
462 Structured 3D Gaussians for view-adaptive rendering. In *IEEE/CVF Conference on Computer
463 Vision and Pattern Recognition (CVPR)*, 2024b. Highlight.

464 Saswat Subhajyoti Mallick, Rahul Goel, Bernhard Kerbl, Markus Steinberger, Francisco Vicente
465 Carrasco, and Fernando De La Torre. Taming 3DGS: High-quality radiance fields with limited
466 resources. In *SIGGRAPH Asia 2024 Conference Papers*, 2024. doi: 10.1145/3680528.3687694.

467 Seungtae Nam, Xiangyu Sun, Gyeongjin Kang, Younggeun Lee, Seungjun Oh, and Eunbyung
468 Park. Generative densification: Learning to densify Gaussians for high-fidelity generalizable 3d
469 reconstruction, 2024.

470 Alexander Novikov, Ngan Vu, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wag-
471 ner, Sergei Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan
472 Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Push-
473 meet Kohli, and Matej Balog. AlphaEvolve: A coding agent for scientific and algorithmic discovery.
474 *arXiv preprint arXiv:2506.13131*, 2025.

475 Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese,
476 Nat McAleese, and Geoffrey Irving. Red teaming language models with language models. In
477 *Empirical Methods in Natural Language Processing (EMNLP)*, 2022.

478 Lukas Radl, Michael Steber, Mathias Parger, Bernhard Kerbl, and Markus Steinberger. StopThePop:
479 Sorted Gaussian Splatting for view-consistent real-time rendering. *ACM Transactions on Graphics*,
480 43(4), 2024. doi: 10.1145/3658187.

481 Kerui Ren, Lihan Jiang, Tao Lu, Mulin Yu, Linning Xu, Zhangkai Ni, and Bo Dai. Octree-GS:
482 Towards consistent real-time rendering with LOD-structured 3D Gaussians. *IEEE Transactions on
483 Pattern Analysis and Machine Intelligence (TPAMI)*, 2025.

484 Shiwei Ren, Tianci Wen, Yongchun Fang, and Biao Lu. FastGS: Training 3D Gaussian Splatting
485 in 100 seconds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*,
486 2026. Highlight.

487 Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog,
488 M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang,
489 Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program
490 search with large language models. *Nature*, 625:468–475, 2024.

491 Samuel Schmidgall and Michael Moor. AgentRxiv: Towards collaborative autonomous research.
492 *arXiv preprint arXiv:2503.18102*, 2025.

493 Samuel Schmidgall, Yusheng Su, Ze Wang, Ximeng Sun, Jialian Wu, Xiaodong Yu, Jiang Liu,
494 Zicheng Liu, and Emad Barsoum. Agent Laboratory: Using LLM agents as research assistants.
495 *arXiv preprint arXiv:2501.04227*, 2025.

496 Chenglei Si, Diyi Yang, and Tatsunori Hashimoto. Can LLMs generate novel research ideas? A
497 large-scale human study with 100+ NLP researchers. In *The Thirteenth International Conference
498 on Learning Representations (ICLR)*, 2025. URL <https://arxiv.org/abs/2409.04109>.

499 Matthew Tancik, Ethan Weber, Evonne Ng, Ruilong Li, Brent Yi, Terrance Wang, Alexander
500 Kristoffersen, Jake Austin, Kamyar Salahi, Abhik Ahuja, et al. Nerfstudio: A modular framework
501 for neural radiance field development. In *ACM SIGGRAPH*, 2023.

502 Jiabin Tang, Lianghao Xia, and Chao Huang. AI-Researcher: Autonomous scientific innovation.
503 *arXiv preprint arXiv:2505.18705*, 2025.

504 Bin Wang, Chao Xu, Xiaomeng Zhao, Linke Ouyang, Fan Wu, Zhiyuan Zhao, Rui Xu, Kaiwen Liu,
505 Yuan Qu, Fukai Shang, Bo Zhang, Liqun Wei, Zhihao Sui, Wei Zhu, Kai Liu, Rui Wu, Conghui
506 Jin, Bo Peng, Yu Dai, and Conghui He. MinerU: An open-source solution for precise document
507 content extraction. *arXiv preprint arXiv:2409.18839*, 2024.

508 Penguin Wang, Yueji Wang, Dongjin Wang, Shijia Mohan, Zhiwen Fan, Liang Wu, Ruijin Cai,
509 Yung-Yu Yeh, Zhangyang Wang, Qixing Liu, and Rakesh Ranjan. Steepest descent density control
510 for compact 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern
511 Recognition (CVPR)*, 2025.

512 Yixuan Weng, Minjun Zhu, Guangsheng Bao, Hongbo Zhang, Jindong Wang, Yue Zhang, and Linyi
513 Yang. CycleResearcher: Improving automated research via automated review. *arXiv preprint
514 arXiv:2411.00816*, 2024.

515 Haofei Xu, Songyou Peng, Fangjinhua Wang, Hermann Blum, Daniel Barath, Andreas Geiger, and
516 Marc Pollefeys. DepthSplat: Connecting Gaussian Splatting and depth. In *IEEE/CVF Conference
517 on Computer Vision and Pattern Recognition (CVPR)*, 2025.

518 Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune,
519 and David Ha. The AI Scientist-v2: Workshop-level automated scientific discovery via agentic
520 tree search. *arXiv preprint arXiv:2504.08066*, 2025.

521 Zhiwen Yan, Weng Fei Low, Yu Chen, and Gim Hee Lee. Multi-scale 3D Gaussian Splatting for
522 anti-aliased rendering. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition
523 (CVPR)*, 2024.

524 Ziyi Yang, Xinyu Gao, Yangtian Sun, Yihua Huang, Xiaoyang Lyu, Wen Zhou, Shaohui Jiao,
525 Xiaojuan Qi, and Xiaogang Jin. Spec-Gaussian: Anisotropic view-dependent appearance for 3D
526 Gaussian Splatting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

527 Haoran Ye, Jiarui Wang, Zhiguang Cao, Federico Berto, Chuanbo Hua, Haeyeon Kim, Jinkyoo Park,
528 and Guojie Song. ReEvo: Large language models as hyper-heuristics with reflective evolution. In
529 *Advances in Neural Information Processing Systems (NeurIPS)*, 2024a.

530 Vickie Ye, Matias Turkulainen, and the nerfstudio team. gsplat: An open-source library for Gaussian
531 Splatting. *Journal of Machine Learning Research (JMLR)*, 26:1–15, 2025.

532 Zongxin Ye, Wenyu Li, Sidun Liu, Peng Qiao, and Yong Dou. AbsGS: Recovering fine details for
533 3D Gaussian Splatting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition
534 (CVPR)*, 2024b.

535 Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-Splatting: Alias-free
536 3D Gaussian splatting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and
537 Pattern Recognition (CVPR)*, pages 19447–19456, 2024.

538 Jiahui Zhang, Fangneng Zhan, Muyu Xu, Shijian Lu, and Eric Xing. FreGS: 3D Gaussian Splatting
539 with progressive frequency regularization. In *IEEE/CVF Conference on Computer Vision and
540 Pattern Recognition (CVPR)*, 2024a.

541 Jiawei Zhang, Jiahe Li, Xiaohan Yu, Lei Huang, Lin Gu, Jin Zheng, and Xiao Bai. CoR-GS: Sparse-
542 view 3D Gaussian Splatting via co-regularization. In *European Conference on Computer Vision
543 (ECCV)*, 2024b.

544 Zheng Zhang, Wenbo Hu, Yixing Lao, Tong He, and Hengshuang Zhao. Pixel-GS: Density control
545 with pixel-aware gradient for 3D Gaussian Splatting. In *European Conference on Computer Vision
546 (ECCV)*, 2024c.

547 Zehao Zhu, Zhiwen Fan, Yifan Jiang, and Zhangyang Wang. FSGS: Real-time few-shot view
548 synthesis using Gaussian Splatting. In *European Conference on Computer Vision (ECCV)*, 2024.

NeurIPS Paper Checklist

The checklist is designed to encourage best practices for responsible machine learning research, addressing issues of reproducibility, transparency, research ethics, and societal impact. Do not remove the checklist: **The papers not including the checklist will be desk rejected.** The checklist should follow the references and follow the (optional) supplemental material. The checklist does NOT count towards the page limit.

Please read the checklist guidelines carefully for information on how to answer these questions. For each question in the checklist:

- You should answer [Yes], [No], or [N/A].
- [N/A] means either that the question is Not Applicable for that particular paper or the relevant information is Not Available.
- Please provide a short (1–2 sentence) justification right after your answer (even for [N/A]).

The checklist answers are an integral part of your paper submission. They are visible to the reviewers, area chairs, senior area chairs, and ethics reviewers. You will also be asked to include it (after eventual revisions) with the final version of your paper, and its final version will be published with the paper.

The reviewers of your paper will be asked to use the checklist as one of the factors in their evaluation. While [Yes] is generally preferable to [No], it is perfectly acceptable to answer [No] provided a proper justification is given (e.g., error bars are not reported because it would be too computationally expensive” or “we were unable to find the license for the dataset we used”). In general, answering [No] or [N/A] is not grounds for rejection. While the questions are phrased in a binary way, we acknowledge that the true answer is often more nuanced, so please just use your best judgment and write a justification to elaborate. All supporting evidence can appear either in the main paper or the supplemental material, provided in appendix. If you answer [Yes] to a question, in the justification please point to the section(s) where related material for the question can be found.

IMPORTANT, please:

- **Delete this instruction block, but keep the section heading “NeurIPS Paper Checklist”.**
- **Keep the checklist subsection headings, questions/answers and guidelines below.**
- **Do not modify the questions and only use the provided macros for your answers.**

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not include experiments.

- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer **[Yes]** if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.

- 753 • The paper should disclose whether the full research project required more compute
754 than the experiments reported in the paper (e.g., preliminary or failed experiments that
755 didn't make it into the paper).

756 **9. Code of ethics**

757 Question: Does the research conducted in the paper conform, in every respect, with the
758 NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

759 Answer: **[TODO]**

760 Justification: **[TODO]**

761 Guidelines:

- 762 • The answer [N/A] means that the authors have not reviewed the NeurIPS Code of
763 Ethics.
764 • If the authors answer [No], they should explain the special circumstances that require a
765 deviation from the Code of Ethics.
766 • The authors should make sure to preserve anonymity (e.g., if there is a special consid-
767 eration due to laws or regulations in their jurisdiction).

768 **10. Broader impacts**

769 Question: Does the paper discuss both potential positive societal impacts and negative
770 societal impacts of the work performed?

771 Answer: **[TODO]**

772 Justification: **[TODO]**

773 Guidelines:

- 774 • The answer [N/A] means that there is no societal impact of the work performed.
775 • If the authors answer [N/A] or [No], they should explain why their work has no societal
776 impact or why the paper does not address societal impact.
777 • Examples of negative societal impacts include potential malicious or unintended uses
778 (e.g., disinformation, generating fake profiles, surveillance), fairness considerations
779 (e.g., deployment of technologies that could make decisions that unfairly impact specific
780 groups), privacy considerations, and security considerations.
781 • The conference expects that many papers will be foundational research and not tied
782 to particular applications, let alone deployments. However, if there is a direct path to
783 any negative applications, the authors should point it out. For example, it is legitimate
784 to point out that an improvement in the quality of generative models could be used to
785 generate Deepfakes for disinformation. On the other hand, it is not needed to point out
786 that a generic algorithm for optimizing neural networks could enable people to train
787 models that generate Deepfakes faster.
788 • The authors should consider possible harms that could arise when the technology is
789 being used as intended and functioning correctly, harms that could arise when the
790 technology is being used as intended but gives incorrect results, and harms following
791 from (intentional or unintentional) misuse of the technology.
792 • If there are negative societal impacts, the authors could also discuss possible mitigation
793 strategies (e.g., gated release of models, providing defenses in addition to attacks,
794 mechanisms for monitoring misuse, mechanisms to monitor how a system learns from
795 feedback over time, improving the efficiency and accessibility of ML).

796 **11. Safeguards**

797 Question: Does the paper describe safeguards that have been put in place for responsible
798 release of data or models that have a high risk for misuse (e.g., pre-trained language models,
799 image generators, or scraped datasets)?

800 Answer: **[TODO]**

801 Justification: **[TODO]**

802 Guidelines:

- 803 • The answer [N/A] means that the paper poses no such risks.

- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: **[TODO]**

Justification: **[TODO]**

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [TODO]

Justification: [TODO]

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

